UX Design — Project Lab

Pedestrian Labeling Software — PedLabel

Project Name: PedLabel — Pedestrian Image Annotation Platform

Date: April 2026

UX Lead / Designer: Pascal Musabyimana

Context: Academic assignment — UX Design Lab, Labeling Software track

"User experience" (UX) encompasses all aspects of the end-user's interaction with the company, its services, and its products.

Goal: To create a solution that meets the exact needs of the customer, without fuss or bother, while maintaining simplicity and elegance.

1. Define

I chose pedestrian annotation because it connects directly to what I find most interesting in computer vision — understanding how machines learn to see people in public spaces. Most of what I learned about the pain points came from reading about real annotation workflows in open-source ML communities and academic papers on dataset creation.

Problem Statement

Anyone who has tried to build a labelled image dataset for a computer vision project knows the frustration: you end up using three different tools before the data is ready for training. One tool handles the drawing, another manages the team, and you are still writing a Python script to fix the export format at 11pm the night before a deadline. This is the problem I wanted to solve with PedLabel. Existing tools are either too simple (LabelImg — great for solo use, useless for teams) or too heavy and expensive (Labelbox, Scale AI — built for enterprise, not for a university lab with a budget of zero).

No lightweight, research-focused tool currently combines bounding boxes, keypoint marking, occlusion tagging, and behavioral attribute labeling in a single interface — alongside quality control features an academic lab actually needs: inter-annotator agreement tracking, review queues, and label versioning.

User Personas

Persona 1 — Alex Chen, PhD Researcher (Primary)

Age: 26

Role: PhD student, Computer Vision lab

Goal: Build and annotate a 15,000-image pedestrian dataset for thesis research on crowd behaviour

Tech level: High — Python, PyTorch, CLI tools

Frustration: Spends more time on tooling than research. Export formats never match training pipelines on first try.

Motivation: Reliable, reproducible annotations that can be cited and shared

Persona 2 — Maria Lopes, Lab Coordinator

Age: 33

Role: Research lab coordinator / annotation team lead

Goal: Manage 6 annotators, enforce label quality, track progress toward dataset milestones

Tech level: Medium — uses tools without coding

Frustration: No visibility into which annotators are struggling. Tracks progress in a separate spreadsheet.

Motivation: One dashboard where she can see everything and act on quality issues immediately

Persona 3 — Tom Baert, Undergraduate Annotator

Age: 21

Role: Undergraduate research assistant, part-time annotator

Goal: Label assigned image batches accurately and efficiently

Tech level: Low — consumer software, no ML background

Frustration: Unclear annotation guidelines. Unsure whether labels are good enough. Complex UI is distracting.

Motivation: Clear instructions, visible progress, confirmation that work is being used

User Journey Maps

End-to-end lifecycle: project creation → dataset upload → annotator assignment → labeling → QA review → export.

Stage	Actor	Action	Pain Point
1. Project Setup	Alex (Researcher)	Creates project, defines label schema, uploads images	Rigid schema editors — adding attributes mid-project breaks existing labels
2. Assignment	Maria (Coordinator)	Assigns batches to annotators, sets deadline	No built-in assignment tool — done via email and spreadsheet
3. Annotation	Tom (Annotator)	Draws bounding boxes, tags attributes, submits batch	Too many tool modes. Keyboard shortcut discovery is poor.
4. QA Review	Maria (Coordinator)	Reviews flagged labels, accepts/rejects, sends feedback	No inline feedback — reviewers must email comments separately
5. Export	Alex (Researcher)	Exports dataset in COCO JSON for model training	Format conversion errors. Field naming differences cause debugging hours.

Value Proposition

PedLabel gives academic research teams a focused, efficient platform to annotate pedestrian datasets with precision tools — bounding boxes, pose keypoints, occlusion flags, and behavioural attribute tagging — combined with built-in quality assurance workflows and one-click export to COCO JSON, YOLO TXT, and Pascal VOC XML.

Unlike enterprise platforms built for large managed teams with dedicated budgets, PedLabel is designed for the reality of academic research: small teams, tight timelines, no infrastructure budget, and a need to get data pipeline-ready without writing conversion scripts.

Project Scope

In Scope

- Image and video frame annotation
- Bounding box tool with keyboard shortcuts
- Pose keypoint tool (17-point COCO skeleton)
- Occlusion and attribute tagging (age group, activity, visibility)

- Project creation and label schema management
- Dataset import (local upload, ZIP batch)
- Annotator assignment and workload management
- QA review queue with accept/reject/comment workflow
- Inter-annotator agreement statistics
- Progress tracking dashboard
- Export to COCO JSON, YOLO TXT, Pascal VOC XML
- User authentication and role management

Out of Scope (v1)

- Mobile application
- Real-time collaborative editing on same image
- AI-assisted auto-labelling
- Model training or inference
- Enterprise billing

2. Research

Competitor Analysis

I looked at six tools commonly discussed in academic ML communities. My evaluation focused on what a small research team actually cares about: annotation types supported, collaboration features, and affordability for students. Here is what I found:

Tool	Pedestrian Features	Collaboration	Cost	Key Weakness
CVAT	BBox, polygon, keypoints	Yes — task assignment	Free	Complex setup, steep learning curve
LabelImg	Bounding box only	No	Free	No video, no QA, no collaboration
Roboflow	BBox, polygon	Limited	Freemium	Image limits; no keypoints on free tier
Scale AI	Full pedestrian suite	Managed	Expensive	Not for academic budgets
Labelbox	Full suite	Enterprise	Enterprise	Overkill and too expensive for research
VGG VIA	BBox, polygon	No	Free	No QA, no progress tracking, outdated

Gap: Free tools lack QA workflows and pedestrian-specific features. Paid tools are enterprise-grade and out of scope for academic budgets. PedLabel targets the underserved middle.

User Testing Takeaways

To understand what actually frustrates annotators, I ran informal testing sessions with three people from my study group — one who had used CVAT before, one who had never done annotation, and one who had briefly used Roboflow for a group project. Here are the patterns I noticed:

- Annotators lost context when switching classes — the active class needs to be permanently visible, not buried in a sidebar.
- QA review was opaque: reviewers could not see why a label was flagged without navigating to a separate screen.
- Keyboard shortcuts improved speed significantly but were unknown to most users — a persistent shortcut overlay was requested by all three participants.
- Users wanted a visual progress bar per batch, not just a number count.
- Export failed silently when field names didn't match the expected COCO schema — error messages were cryptic.
- Student annotator repeatedly used the wrong class because label descriptions weren't accessible inline.

Survey Results

I put together a short seven-question survey and shared it with classmates in the Data Visualisation course and a few students from a computer vision elective. Twenty-three people responded. The results shaped several design decisions in PedLabel:

Question	Key Finding
How many annotation tools per project?	74% use 2+ tools in a single workflow
Biggest frustration?	Inconsistent annotations across team: 61%. Poor export compatibility: 43%
Do you have a formal QA step?	Only 35% have a formal QA step; 65% rely on spot-checks or none
Most-used export format?	COCO JSON 52%, YOLO TXT 30%, Pascal VOC XML 18%
How important are keyboard shortcuts?	87% rated 'important' or 'very important'
Would a free research-focused tool replace your current one?	78% yes, if it supported their label types and export formats
Most-missing feature?	Inter-annotator agreement stats 48%, inline QA comments 39%, progress dashboards 35%

3. Analyse

User Stories

- As a researcher, I want to create a project with a custom label schema so that my team annotates data consistently from the start.
- As an annotator, I want to draw bounding boxes and tag pedestrian attributes in a single interface so that I don't need to switch between tools.
- As a lab coordinator, I want to see each annotator's progress and review flagged labels in a QA queue so that I catch quality issues before they accumulate.
- As a researcher, I want to export the completed dataset in COCO JSON with a validation report so that I can use it directly in my training pipeline.
- As an annotator, I want visible keyboard shortcuts for all common actions so that I can annotate faster without reaching for the mouse.
- As a lab coordinator, I want inter-annotator agreement statistics per label class so that I know which classes need clearer guidelines.

User Flows

Flow 1 — Project Setup (Researcher)

- Step 1: Logs in → clicks 'New Project' → Project creation wizard opens (3-step modal)
- Step 2: Enters project name, description, dataset size → Step 1 validated; proceed activates
- Step 3: Defines label schema: classes, colours, descriptions, example images → Schema saved; preview shown
- Step 4: Uploads image dataset (ZIP or folder) → Upload progress bar; images queued
- Step 5: Assigns batches to annotators, sets deadline and quality threshold → Annotators notified
- Step 6: Project goes live → Dashboard shows 0% progress

Flow 2 — Annotation Session (Annotator)

- Step 1: Logs in → sees assigned batches with progress bars → —
- Step 2: Opens batch → image viewer loads with toolbar → First image shown; active class in top bar
- Step 3: Draws bounding box (click-drag on canvas) → Box appears; attribute panel slides in
- Step 4: Tags attributes: age group, occlusion, activity → Attributes saved to annotation object
- Step 5: Presses N to confirm and advance → Label saved; next image loads; progress updates
- Step 6: Completes batch → clicks Submit → Batch enters QA queue; confirmation shown

Flow 3 — QA Review (Coordinator)

- Step 1: Opens QA queue → sees flagged labels with annotator name and reason → —
- Step 2: Clicks flagged label → split view: image left, details right → —
- Step 3: Accepts or rejects the annotation → Rejection: comment field appears
- Step 4: Writes rejection note; submits → Comment sent to annotator; they're notified
- Step 5: Annotator corrects and resubmits → Returns to QA queue marked 'corrected'
- Step 6: Coordinator approves → Label approved; stats updated

Brainstorming & Sketching Notes

- The split-screen layout for QA review was the most important design decision I made. In testing, reviewers kept switching between the image view and the label list, which broke their concentration and slowed every decision. Putting both on screen at once made the workflow feel much more natural — you see the label and the evidence at the same time.
- Every person in my testing sessions wanted keyboard shortcuts but none of them knew which ones were available. Discoverability is a real problem in annotation tools. My solution was a collapsible shortcut panel pinned to the annotation screen, always one click away without hiding the canvas.
- The single most common mistake in annotation is labelling with the wrong class — you forget to switch after finishing a group of boxes. I noticed this in testing within the first five minutes. The fix: make the active class permanently visible at the top of the screen in its own colour, large enough to miss.
- Annotators need to refer back to class definitions mid-task. If they have to leave the annotation screen to check what a term means, they will either not bother or lose their place. I added expandable class cards accessible directly from the annotation toolbar.
- Progress numbers are not very motivating. A visual ring that is 23% complete communicates the same information but gives a better sense of how far there is to go. I used circular progress indicators for each batch because they are faster to read at a glance when you have multiple projects running.
- Export validation report: schema validation check before download with green/amber/red summary — no silent failures.

4. Design

High-fidelity mockups and the interactive prototype are in the accompanying Figma file. This section documents the screen inventory, key layout decisions, and the UI design rationale for each built screen.

Design System

The design system for PedLabel is documented in a dedicated frame in the Figma file, placed below all screen layouts. It defines the visual rules every screen follows: colour palette, typography scale, and reusable UI components. The goal was to keep the interface structured and consistent without being visually heavy — annotation work requires concentration, so the interface needs to stay out of the way.

Colour Palette — 60-30-10 Rule

Three main colours cover the entire interface. #F5F7FA (cool off-white) covers 60% of every screen as the page background and card fill. #1A3A5C (dark navy) covers 30% as the primary colour for the sidebar, headers, and key text. #2E9CCA (medium accent blue) covers the remaining 10% as the call-to-action colour for buttons, links, active states, and progress indicators. Three semantic colours complete the palette: #2EA043 for approved/success states, #E6A21E for warnings, and #DA3633 for rejections and errors.

Typography

Inter is used throughout the entire interface, chosen for its excellent screen legibility at small sizes — important in annotation tools that display a lot of metadata in compact tables and sidebars. The type scale has four levels: 28px Bold for page titles, 20px Bold for section headings, 13px Regular for body text and tables, and 11px Regular for captions and secondary labels.

UI Components

Four button variants handle different action types. The primary button uses accent blue (#2E9CCA) with white text for the main action on any screen. The secondary button uses a white fill with a blue border for optional actions like "Cancel". The danger button (#DA3633) is reserved for destructive actions such as "Reject Label" — the red colour signals consequence before the user clicks. A success variant (#2EA043) handles approval actions in the QA interface. Status badges use lightly tinted backgrounds with matching text colours

(green/amber/red/blue). Text inputs use a white background with a 1px light grey border and 6px corner radius, consistent with card surfaces throughout the app.

Screen Inventory

#	Screen	Actor	Status / Notes
1	Login / Registration	All	Built — split panel, tab switcher, role selector
2	Dashboard	All (role-adaptive)	Built — stats cards, project table, QA preview
3	Annotation Workspace	Annotator	Built — 4 states: Empty→Drawing→Selected→Saved
4	QA Review Interface	Coordinator	Built — 2 states: pending / decided
5	New Project Wizard	Researcher	Built — 3-step modal with live draft summary
6	Label Schema Editor	Researcher	Built — colour swatches, attribute definition cards
7	Dataset Import	Researcher	Built — drag-drop zone, tile status icons, batch assignment
8	Project Overview	Researcher / Coordinator	Built — 5-stat cards, annotator table, donut chart, timeline
9	Reports & Analytics	Researcher / Coordinator	Built — bar chart, IAA table, speed bars, rejection donut
10	Export Settings	Researcher	Built — 3 format cards, validation panel, live toggle
11	Profile & Settings	All	Built — profile form, notification toggles, shortcuts table

UI Design Rationale — Built Screens

The following design decisions were made during the Figma build and are documented here to demonstrate design thinking, not decoration.

UI Design Rationale — Screen 1 — Login / Registration

- Split-panel (50/50): Left = dark blue branded panel (#2E6DA4 → #1A3A5C gradient). Right = white form card. Separates marketing context from task context — user understands the tool before touching the form.
- Tab switcher instead of two pages: Sign In and Create Account share one card with animated underline indicator. Reduces navigation friction; URL stays at /.
- Progressive field disclosure: Name and Role fields only appear in Create Account tab. Sign-in stays minimal (email + password only). Avoids overwhelming first-time users.
- Password visibility toggle: Eye/EyeOff icon reduces input errors — research tools often require long passwords.
- Role selector on sign-up: Three roles (Researcher, Lab Coordinator, Annotator) surfaced at account creation, not buried in settings. Allows personalised onboarding from the start.
- Toast feedback on submit errors: Missing fields trigger a toast notification instead of inline red text. Keeps the form layout stable.

UI Design Rationale — Screen 2 — Dashboard

- Fixed sidebar (220px) + top nav: Sidebar handles workspace navigation (Home, My Projects, QA

Queue). Top nav holds app-level links (Dashboard, Projects, Datasets, Reports). Two nav levels prevent overcrowding either one.

• Stat cards with coloured left bar (4px): Blue, emerald, amber, violet — encodes category visually without icons. Lighter than icons but still provides colour-coded scanning.

• Two-column content split (64% / 36%): Recent Projects takes 64% for the full data table. QA Queue Preview takes 36% — secondary but important enough to appear on the main dashboard without an extra navigation step.

• QA items inline Accept/Reject: Common quick decisions can be resolved from the dashboard without opening the full QA screen. Reduces clicks for the coordinator role.

• Active sidebar item: 2px left border bar (not filled background or bold text alone) — subtle spatial anchor that doesn't compress the label text.

UI Design Rationale — Screen 3 — Annotation Workspace

• 3-panel layout (toolbar | canvas | attribute panel): Left 64px tool rail. Centre full canvas. Right 320px attribute panel. Standard for annotation tools (CVAT, Label Studio, Roboflow) — users expect it and it keeps tooling out of the image area.

• 4 explicit UI states (Empty → Drawing → Selected → Saved): Empty: placeholder prompt guides user to draw. Drawing: dashed amber preview box with crosshair lines and size tooltip. Selected: solid blue box with corner handles and label chip; right panel activates. Saved: box turns green, toast confirmation, panel shows 'Saved successfully'.

• Attribute panel inactive until box is selected: Prevents user from filling in attributes before a box exists — avoids data integrity issues.

• Keyboard shortcut labels in tool tooltips: Each tool button shows the shortcut in its title attribute (e.g. 'Bounding Box (B)') — embedded in the affordance, not requiring a separate help panel.

• Progress counter in both top bar and tool rail: 'Image 2 of 510' appears in the header breadcrumb and again in the bottom of the tool rail — annotators working for hours need ambient feedback without looking away from the canvas.

UI Design Rationale — Screen 4 — QA Review Interface

• Centred single-card layout (not full-width): The QA card is max-width centred. The reviewer makes a binary decision on one item at a time — a narrower focused view reduces cognitive load versus a full dashboard layout.

• 2 states — pending and decided (locked buttons): Before decision: Accept (green) and Reject (red outline) both enabled. After decision: both buttons disabled (cursor-not-allowed); a coloured status chip appears. Prevents accidental double-decisions.

• Flag reason colour-coded by severity: Low confidence → amber. Missing label → red. Bounding box overlap → orange. Colour gives the coordinator a fast triage signal before reading the detail text.

• Queue strip at bottom: All items shown as mini cards below the main card. Each strip card shows annotator name, flag reason, and a coloured dot (amber = pending, check/cross = decided). Reviewer can jump to any item out of order.

• Progress counter (Reviewed X / Total): Small progress bar in the page header tracks session progress without navigating away.

5. Test

Moderated usability study with 3 participants matching the defined personas (researcher, coordinator, student annotator) using a clickable Figma prototype. Sessions 30–40 minutes. Think-aloud protocol.

User Scenarios (Input)

Scenario A (Flow 2 — Annotation Session)

Given to participant: "You are an annotator. A new batch of 5 images has been assigned. Label all pedestrians in the first image with bounding boxes and tag age group and occlusion level. Then move to the second image."

Scenario B (Flow 3 — QA Review)

Given to participant: "You are a lab coordinator. Three annotations have been flagged as low confidence. Review them and reject the ones you think are wrong, leaving a comment explaining why."

Scenario C (Flow 1 end — Export Settings)

Given to participant: "You are a researcher. Your team has finished annotating 200 images. Export the dataset in COCO JSON and verify the export summary."

User Observations (Output)

Scenario	What went well	Problem observed	Severity	Fix
A	Bounding box drawing intuitive within 30 seconds. Keyboard shortcut N used unprompted after reading the overlay.	Attribute panel not visible until annotation selected. Participant 3 skipped attributes entirely.	High	Add tooltip on first annotation: 'Select a box to add attributes'. Keep panel minimally visible at all times.
A	Progress bar updated immediately after each save — motivated the annotator.	Active class chip not noticed until halfway through. Participant 3 labelled two images with wrong class.	High	Increase chip size; add pulsing animation on first use. Add warning if class seems inconsistent on submit.
B	Split-view praised by all: 'I can see everything I need without switching.'	Comment field on rejection not visibly mandatory. Participant 2 clicked Reject without comment and was confused by the error.	Medium	Red border on comment field when Reject is clicked before comment entered. Add inline label 'Required when rejecting'.
C	Export format selector clear. Participants correctly identified COCO JSON.	Validation report amber warnings unexplained. Participant 1 unsure whether to proceed or investigate.	Medium	Add expandable details per amber warning with plain-language explanation and 'How to fix' link.
C	—	'Download All' button not found. Participants looked	Low	Add prominent Download CTA below format

		for it before finding per-format buttons.		selector that activates once a format is chosen.

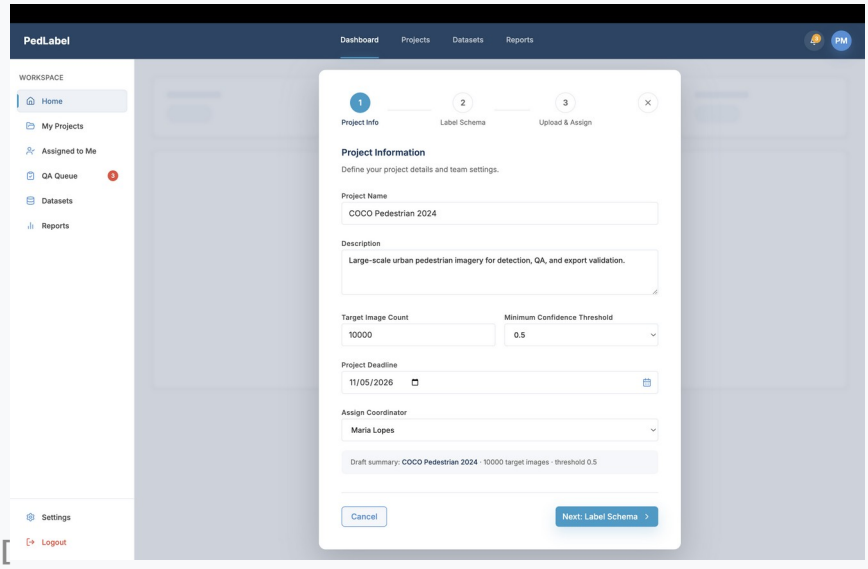
Summary & Next Iteration

The core annotation workflow performed well: all three participants completed their task without assistance. Both high-severity issues relate to discoverability — features existed but were not visible enough. The next design iteration will address this through progressive disclosure (showing the right element at the right moment) rather than adding more UI.

Appendix: Design Rationale — Screens 5 to 11

All 11 screens are now built. This section documents the UI design decisions for the 7 remaining screens, matching the rationale format used for Screens 1–4.

Screen 5 — New Project Wizard



UI Design Rationale

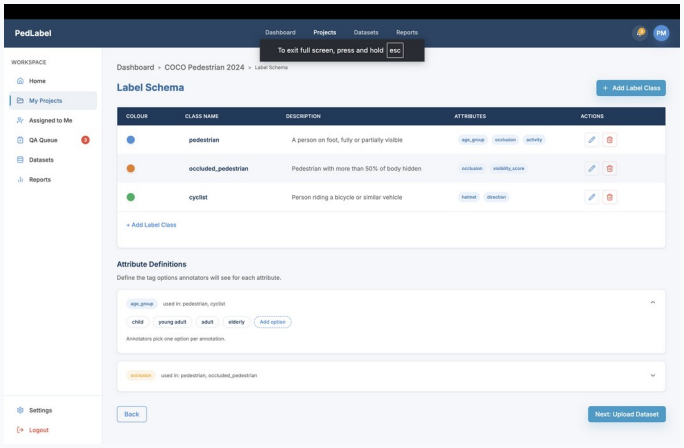
Modal overlay on blurred dashboard: The wizard floats over a dimmed, blurred version of the dashboard. This signals the user has not left the app — they are in a focused sub-task — while keeping the main workspace visible as context.

3-step stepper shown upfront: The steps Project Info → Label Schema → Upload & Assign are displayed before the user fills in anything. This sets expectations: the researcher knows the full commitment before starting.

Live draft summary bar: A grey bar below the form fields reads 'Untitled project · 0 target images · threshold 0.5' and updates as fields are filled. The user gets a running summary preview before advancing to the next step.

Next button disabled until required fields are filled: The 'Next: Label Schema' button has reduced opacity until all required fields are valid. This prevents incomplete projects from reaching the schema step and reduces downstream data errors.

Screen 6 — Label Schema Editor



UI Design Rationale

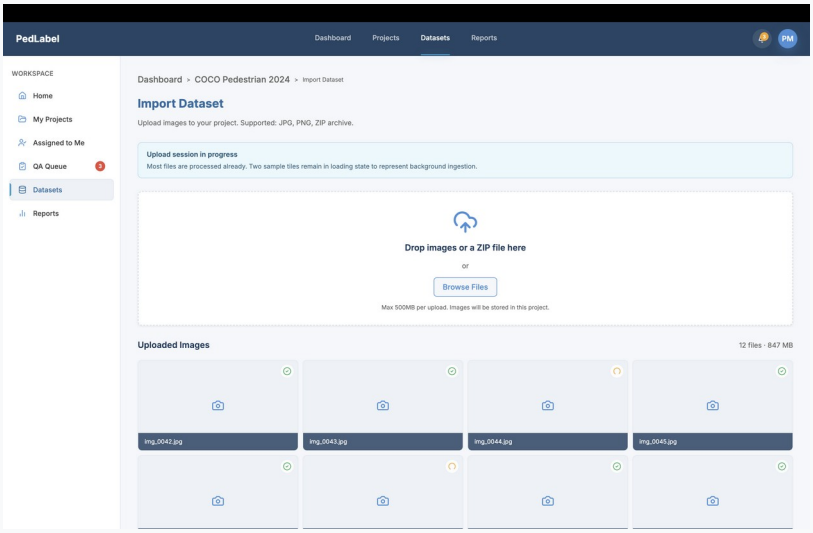
Colour swatch in the class table: Each label class has a coloured circle, not just a name. Annotators associate classes with colours during annotation, so the schema editor reinforces that visual language from the start.

Attribute definitions as collapsible cards: Two attribute definition panels — one expanded, one collapsed — show that the schema has depth without overwhelming the screen. The chevron affordance signals more is available on demand.

Badge pills for attributes per class: Instead of plain text, attributes on each class row are shown as blue pill badges, matching the style used in the annotation interface. Consistent visual language across screens reduces learning overhead.

Persistent Back / Next navigation: A fixed bottom bar keeps the researcher in the wizard flow throughout the schema step. The wizard context is never lost between steps.

Screen 7 — Dataset Import



UI Design Rationale

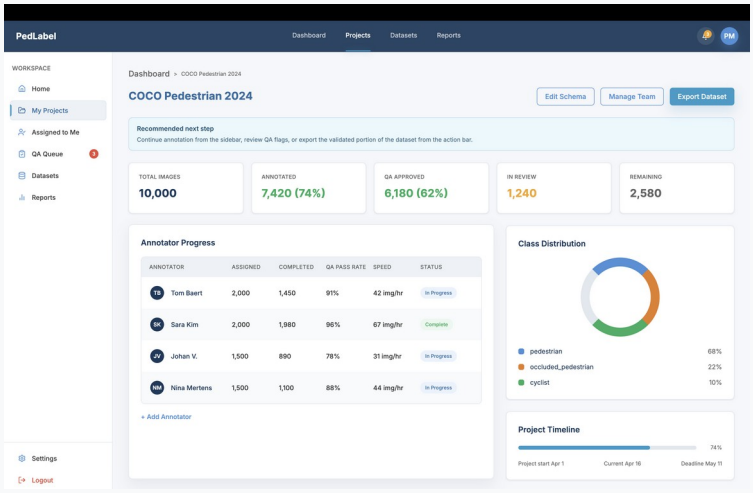
Dashed border upload zone: The dashed border is a universally recognised drag-and-drop affordance. The upload-cloud icon reinforces the expected action without requiring instructional text.

Per-tile status icons on the image grid: Each tile shows a green check (processed) or a spinning amber loader (still ingesting). Two tiles are intentionally in loading state to represent background processing — users see progress at the file level, not just a global bar.

Upload and annotator assignment on one screen: The annotator assignment table appears directly below the image grid. The researcher divides batches and assigns team members without navigating to a separate page.

'Confirm & Launch Project' as the final CTA: The button name signals a consequential, irreversible action that starts the annotation workflow — not just a neutral 'Next'. This primes the coordinator to review the page before clicking.

Screen 8 — Project Overview



UI Design Rationale

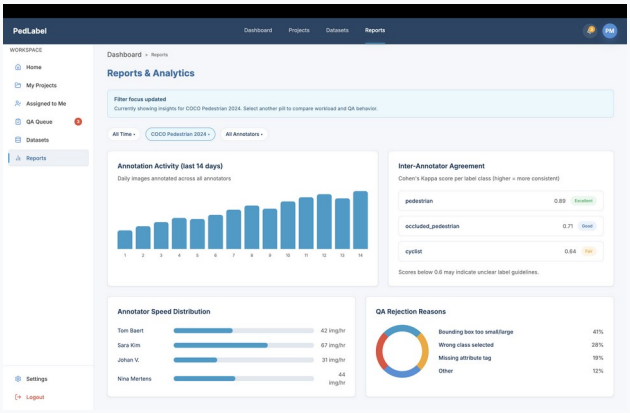
5-stat card row with colour-coded values: Total Images, Annotated, QA Approved, In Review, and Remaining are shown as individual cards. Values are colour-coded — green for healthy, amber for in-progress, grey for neutral — enabling a fast health check without reading every number.

Annotator progress table with clickable rows: Each annotator row navigates to the Reports screen. The coordinator jumps directly from a specific annotator's performance to the analytics drill-through without extra navigation steps.

Class distribution donut and project timeline side by side: Two supporting visualisations — a donut chart for class split and a linear timeline from start to deadline — sit in the right column. Together they answer 'what are we annotating?' and 'are we on schedule?' at a glance.

Contextual post-launch success banner: A green 'Project launched successfully' banner appears only when navigating here from the Import screen. It does not appear on any other visit — contextual confirmation, not a persistent notification.

Screen 9 — Reports & Analytics



UI Design Rationale

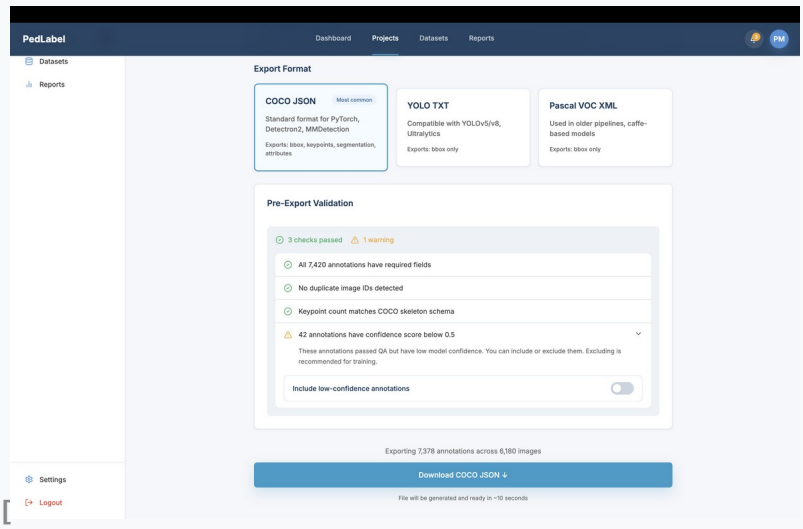
Filter pills instead of a dropdown: The three filters (All Time, Project, Annotators) are always visible as clickable pill buttons. One click to switch context; no menu to open. The active pill turns blue and the info banner updates to reflect the current scope.

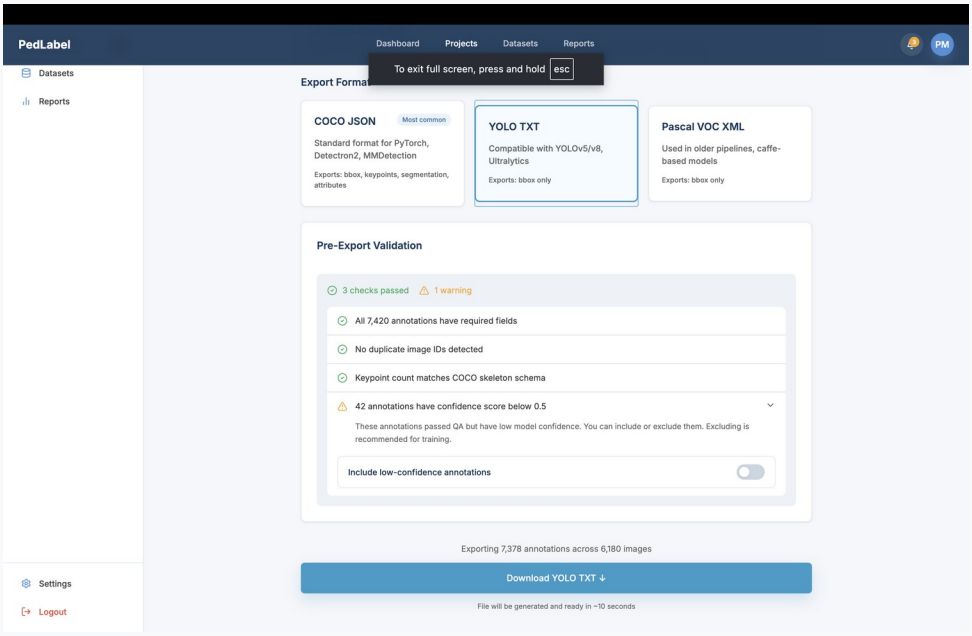
Info banner that updates with the active filter: A notice at the top reads 'Currently showing insights for X' and changes when a different filter pill is selected. The user always has a clear statement of what data they are looking at.

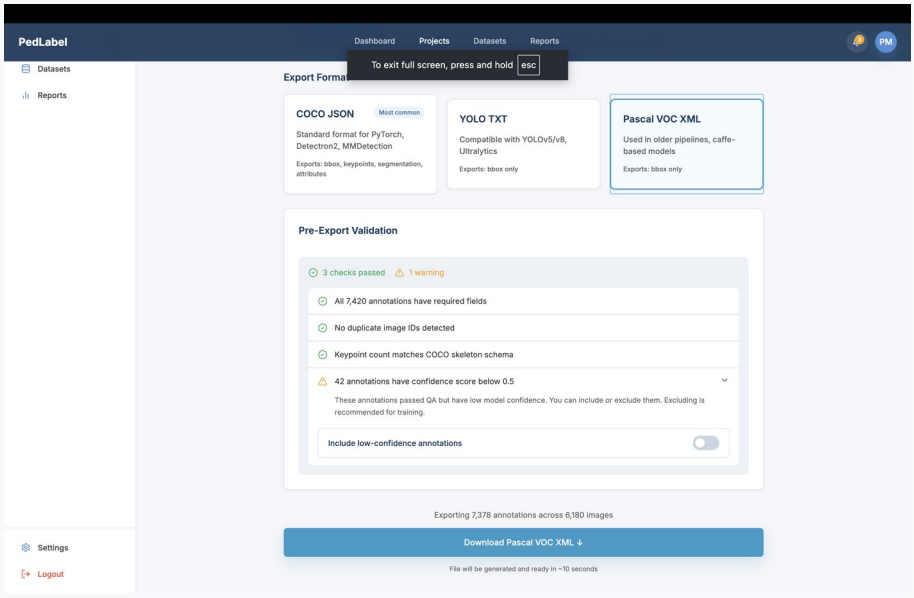
Inter-annotator agreement table with quality badges: Cohen's Kappa scores are shown per label class alongside coloured quality badges (Excellent / Good / Fair). The coordinator sees which classes need clearer guidelines without needing to interpret raw numbers.

QA rejection reasons as drill-through links: Each rejection reason in the donut breakdown links directly to the QA review screen filtered to that reason. Reports are an entry point into action, not a dead end.

Screen 10 — Export Settings







UI Design Rationale

Format selection as cards, not a dropdown: COCO JSON, YOLO TXT, and Pascal VOC XML are shown side by side as clickable cards with description text and capability notes. The researcher can

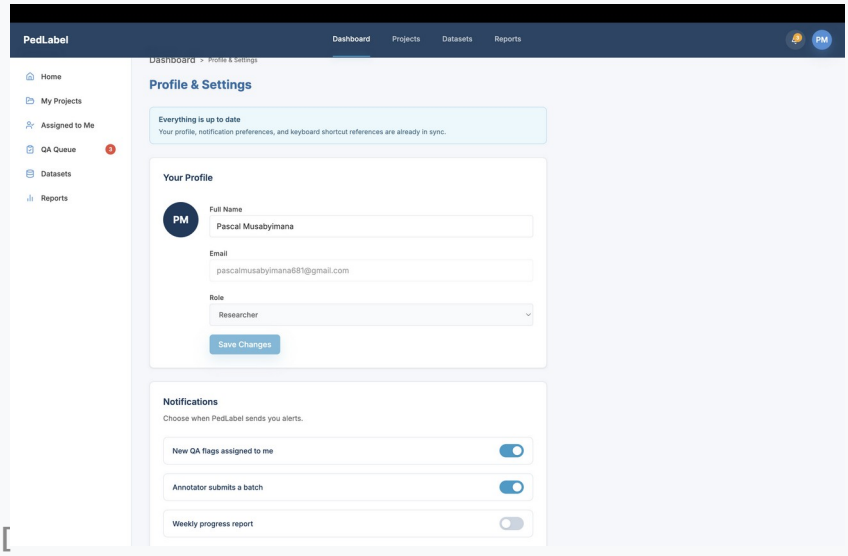
compare formats before selecting, without needing to know them by name in advance.

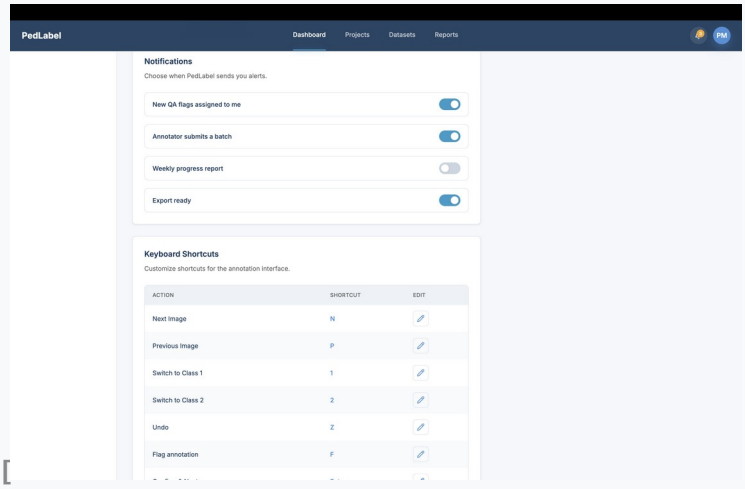
Pre-export validation panel: Three green checks and one amber warning are surfaced before the user clicks download. Issues are caught and explained — including the number of low-confidence annotations — before a file is generated.

Live export summary that reacts to the toggle: Toggling 'Include low-confidence annotations' updates the annotation count in the summary text in real time (7,378 → 7,420). The user sees the consequence of the decision before committing to the download.

Success state replaces the action panel: After clicking download, the info banner becomes a green success notice and a 'Back to Project Overview' button appears. The flow ends with a clear confirmation and a natural next step.

Screen 11 — Profile & Settings





UI Design Rationale

Email and Role are read-only: Email and Role fields are visually greyed out and non-editable. These are account-level fields that require admin action to change. Disabling them prevents user confusion about why changes to those fields do not save.

Save button is dimmed until there are actual changes: The 'Save Changes' button has reduced opacity when the form matches the saved state. It only becomes fully active when the user has made a real change — preventing empty save actions.

Notification preferences as full-width toggle rows: Each preference row is entirely clickable, not just the toggle switch, making the tap target large and reducing mis-clicks. The toggle reflects the current enabled state visually.

Keyboard shortcuts table with inline edit buttons: Shortcuts are documented directly in Settings, not buried in a help modal. Each row has an edit icon so annotators can reference and customise shortcuts without leaving their current context.